Security, Privacy & Adversarial Robustness

1. Prompt injection

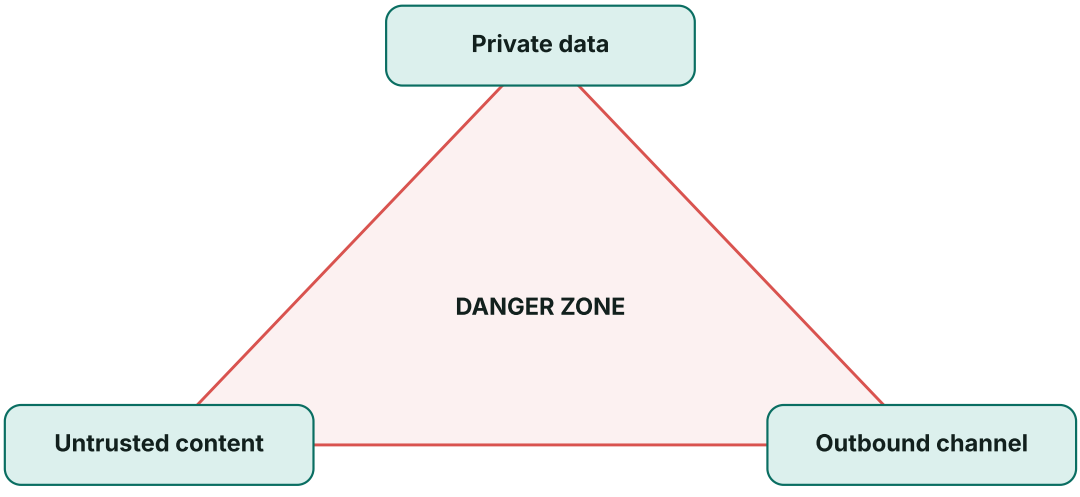
The most important security fact about LLM systems: the model **cannot reliably tell instructions from data**. Anything in the context (a user message, a retrieved document, an email, a web page, a tool result) may be treated as an instruction.

- **Direct injection**: the user tells the model to ignore its rules
- **Indirect injection** (the dangerous one): instructions hidden in content the system reads for the user. A CV telling the screening bot to rank it first; an email telling the assistant to forward the inbox
- As of today **no complete defence exists**. Saying so plainly is a credibility marker; anyone selling a guaranteed fix is selling snake oil

2. The lethal trifecta

Real danger concentrates where a system combines all three: **private data** + **untrusted content** + **a way to act or send outward**. Remove or human-gate any one leg and the catastrophic scenarios collapse. That's why the defences are architectural, not prompt-level.

✎ DIAGRAM



Cut any one side (remove data access, quarantine untrusted reading, or approval-gate the outbound action) and the worst cases go away.

The lethal trifecta.

3. The defence stack, strongest first

1. **Least privilege**: tools get only what the task needs
2. **Human approval** on consequential actions

3 **Privilege separation**: a trusted agent that never reads untrusted content directs a quarantined one that does

4 Treat all tool and retrieval output as **untrusted**

5 **Output / action filtering**: allow-list what agents may fetch and where they may send

6 **Injection classifiers**: useful but porous

7 **Logging and red-teaming**, with attack cases in the eval suite

4. Quieter risks

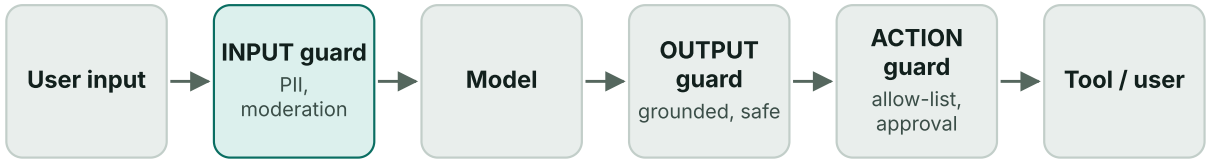
- **Data leakage** from over-permissioned retrieval (a security control, not a nicety)
- Sensitive data sent to third-party APIs without contractual cover
- **Jailbreaks**: mostly brand and abuse risk
- **Supply chain**: every MCP server and plugin is third-party code with data access. Vet it, pin it, limit it

5. Guardrails at input, output and action

- **Input** (before the model sees it): PII detection and redaction, moderation, injection detection, topic limits
- **Output** (before the user sees it): schema validation, **groundedness** (claims backed by sources), PII-leak and toxicity checks, business rules ('never quote prices')
- **Action** (before a tool acts): allow-lists, spend and volume limits, human approval
- Every check adds latency and cost, so **tier them**: cheap rules first, model checks only when needed

> ⚠ Over-strict guardrails make a useless product. Pair every block with a way forward: offer a callback, escalate to a human, point to the right page.

✎ DIAGRAM



Three checkpoints, each with a graceful fallback rather than a dead end.

SAY IT

> *"The defence is architectural: least privilege, human approval for risky actions, and never combining private data, untrusted input and an outbound channel. Guardrails sit at input, output and action, each with a way forward."*